

Strings

Computer Programming 2
2nd Semester 2021-2022

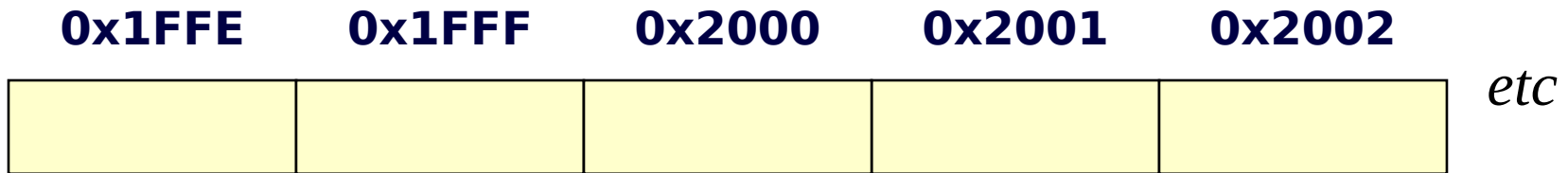


Topics

- Representation
- Declaration
- Index of a char in a string
- String operations
- Common mistakes

Representation

- *Recall*: Main memory
 - contiguous array of cells
 - each cell has an address



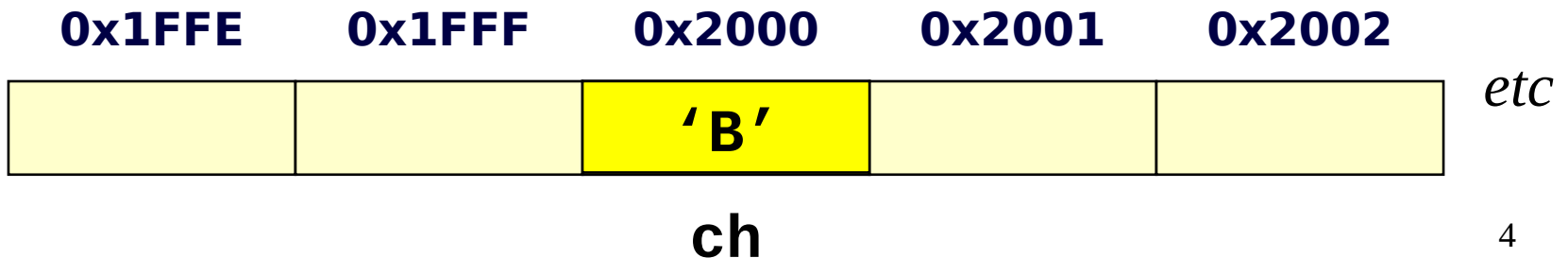
Representation (cont)

- *Recall:* Variable declaration
 - sets aside a “box” to contain a value

Example:

```
char  ch;
```

```
ch = 'B';
```



Representation (cont)

- String declaration
 - sets aside an **array of cells**
 - each cell contains a **char**
 - **address** of first cell in the array

Example:

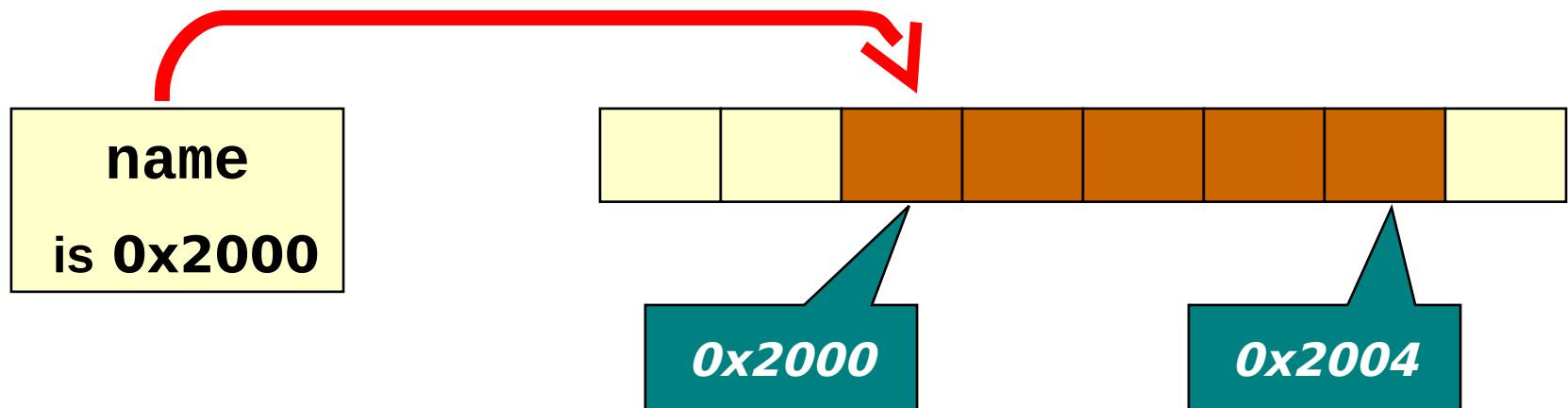
```
char name[5];
```

***Specifies number
of cells in the array***

Representation (cont)

- String declaration
 - sets aside an **array of cells**
 - each cell contains a **char**
 - **address** of first cell in the array

Example: `char name[5];`



Character Arrays vs Character Strings

- A character string is a char array
- A character string *must* have the terminating character (`'\0'`)
- The terminating character allows `scanf()` and `printf()` to handle character strings

Character Strings

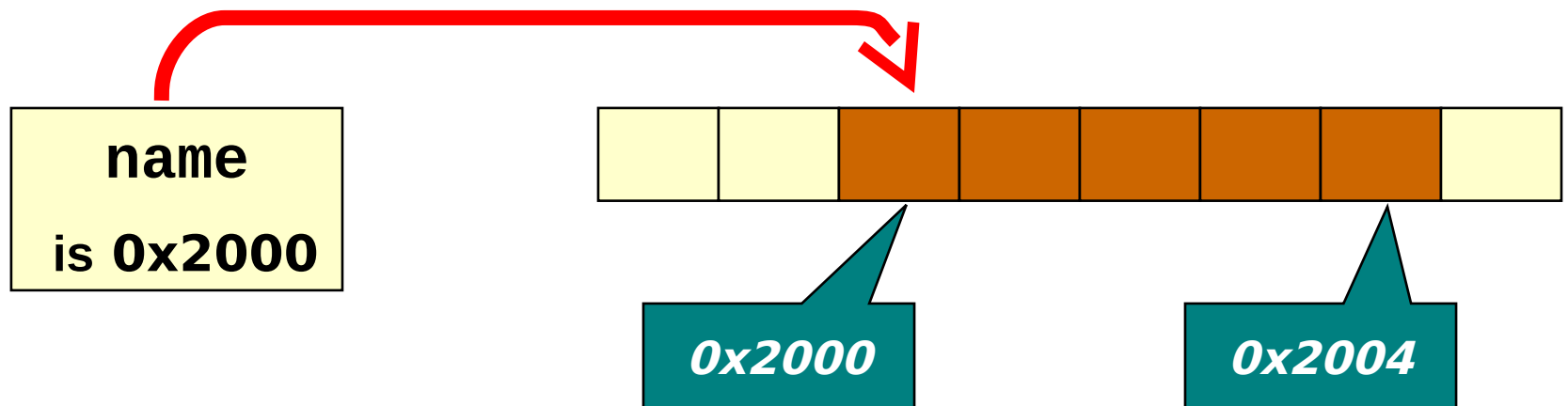
Declaration 1:

```
char  name[5];
```

Declaration 2:

```
#define MAXLENGTH 5
```

```
char  name[MAXLENGTH];
```



String Input/Output

```
#include <stdio.h>
```

```
#define MAXLENGTH 15
```

```
int main()
```

```
{
```

```
    char string1[MAXLENGTH];
```

```
    char string2[MAXLENGTH];
```

```
    scanf("%s %s", string1, string2);
```

```
    printf("%s %s\n", string1, string2);
```

```
    return 0;
```

```
}
```

No ampersand (&)!

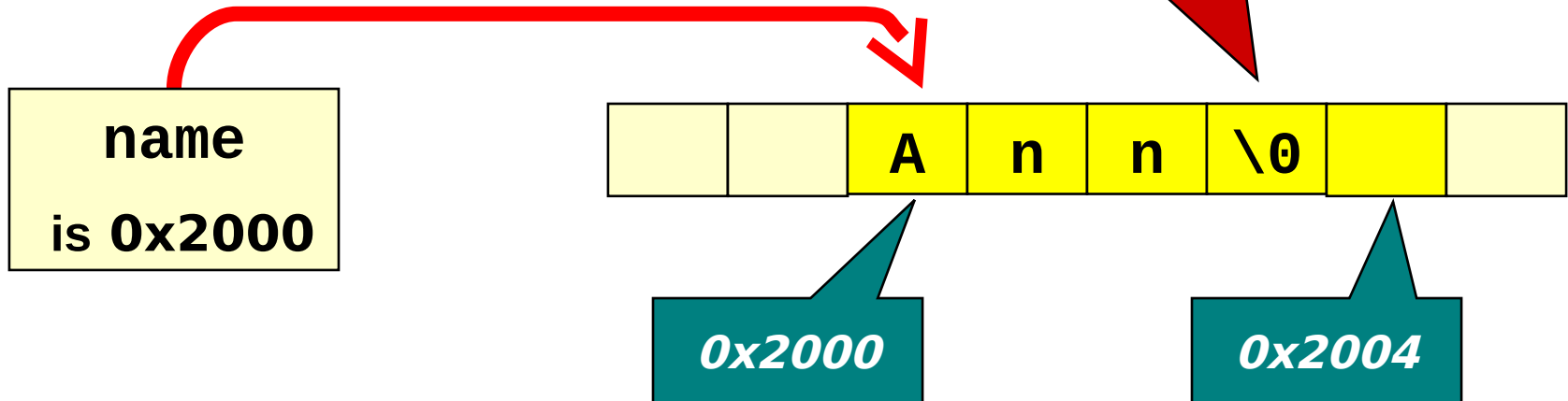
Character String Declaration

Declaration w/ initialization 1:

```
char name[5] = "Ann";
```

Terminating Character:

- Marks the end of string
- Special char: '**\0**'
- aka **NUL** (single L)

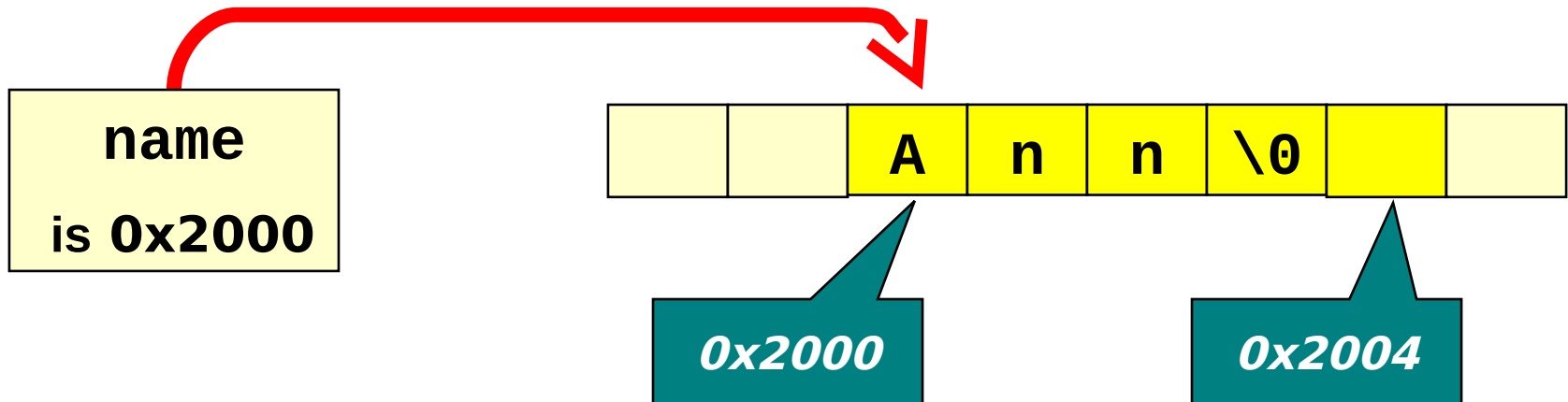


Character String Declaration (cont)

Declaration w/ initialization 1:

```
char name[5] = "Ann";
```

Can store
at most **4 letters**,
because of `'\0'`

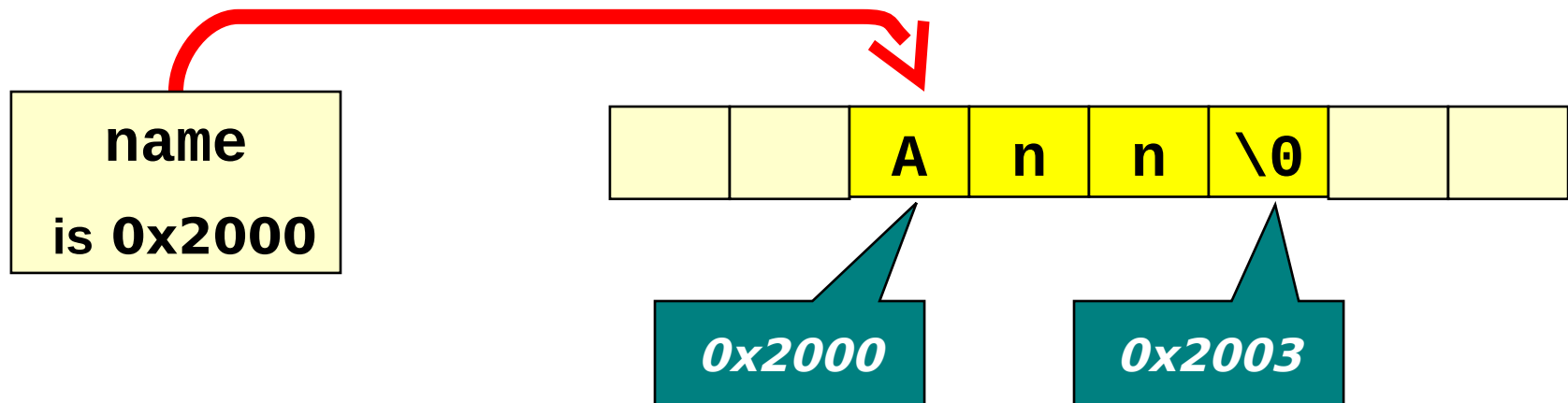


Character String Declaration (cont)

Declaration 2: 

```
char name[] = "Ann";
```

Takes up an extra cell for '\0'

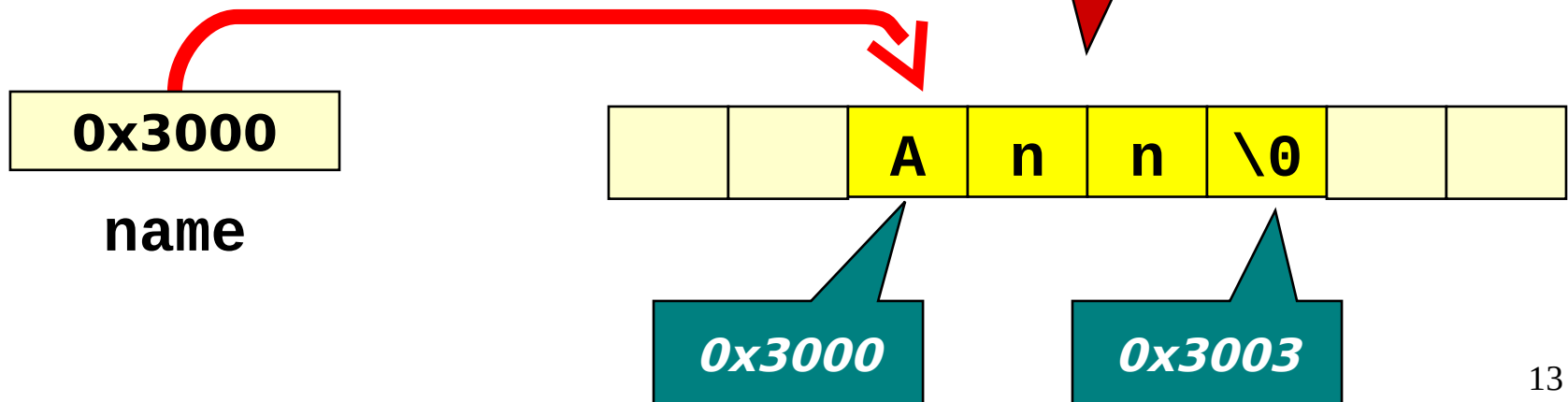


Character String Declaration (cont)

Declaration 3:  

```
char *name = "Ann";
```

**Result is “*undefined*”
if you try to modify
this string**



Character String Declaration (cont)

Declaration 4:



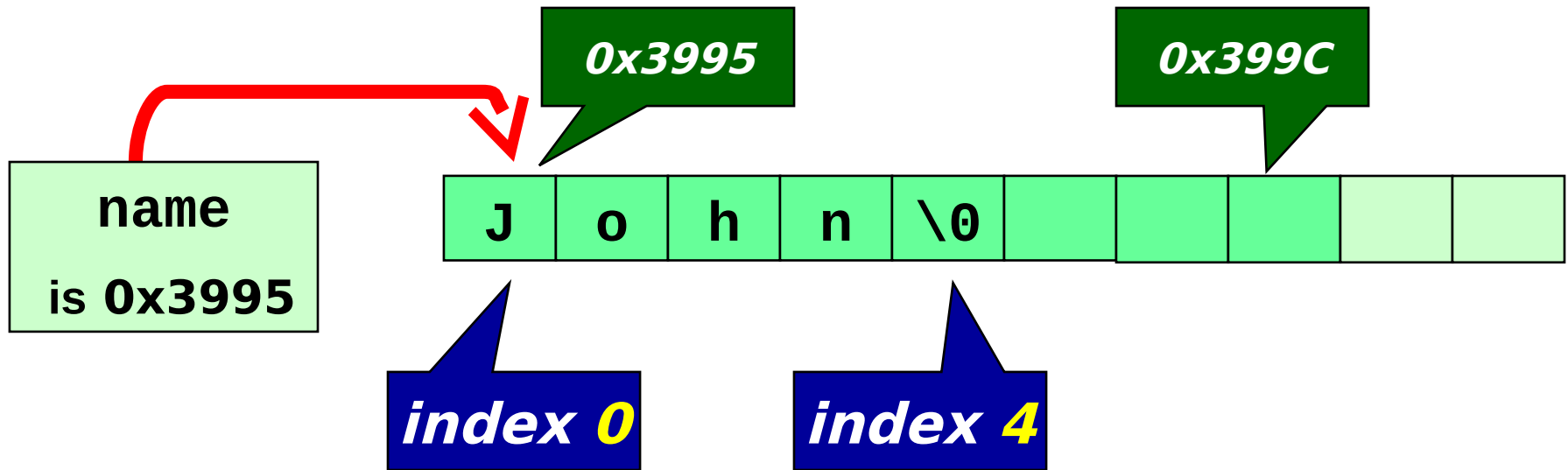
```
char name[];
```

*String with
arbitrary length?
**No! Will cause
an error***

A Char in a String

- The size of a character string is **fixed**
- Character at position *index*:
 - **`string[index]`**
 - first character has index 0

A Char in a String (cont)

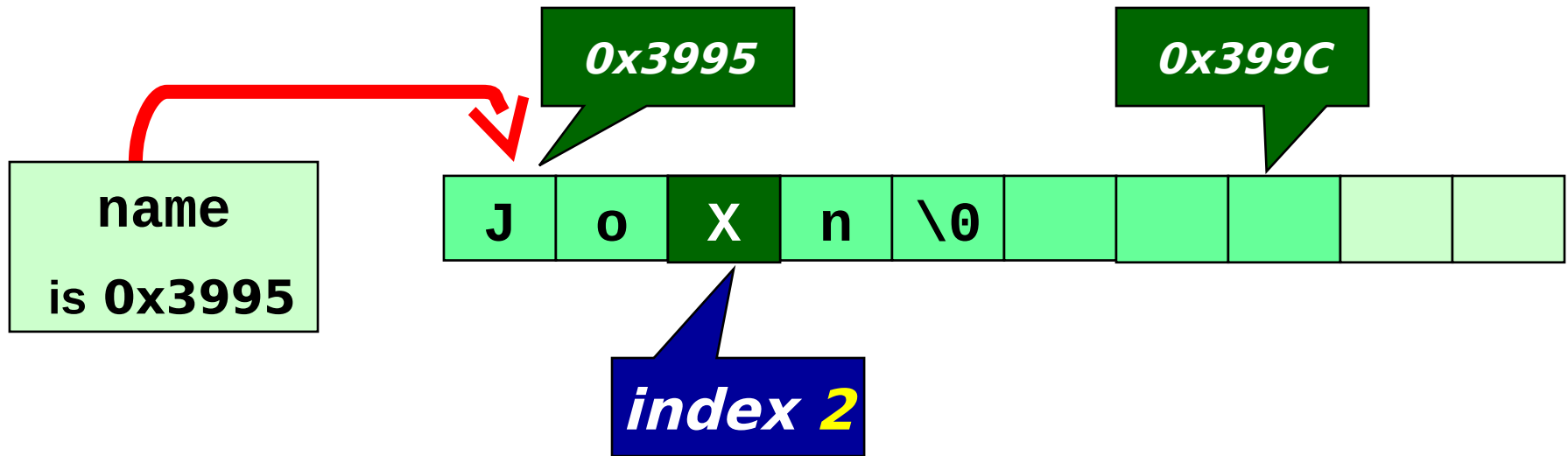


```
char name[8] = "John";  
int i = 2;
```

```
printf("Char at index %d is %c.\n", i, name[i]);
```

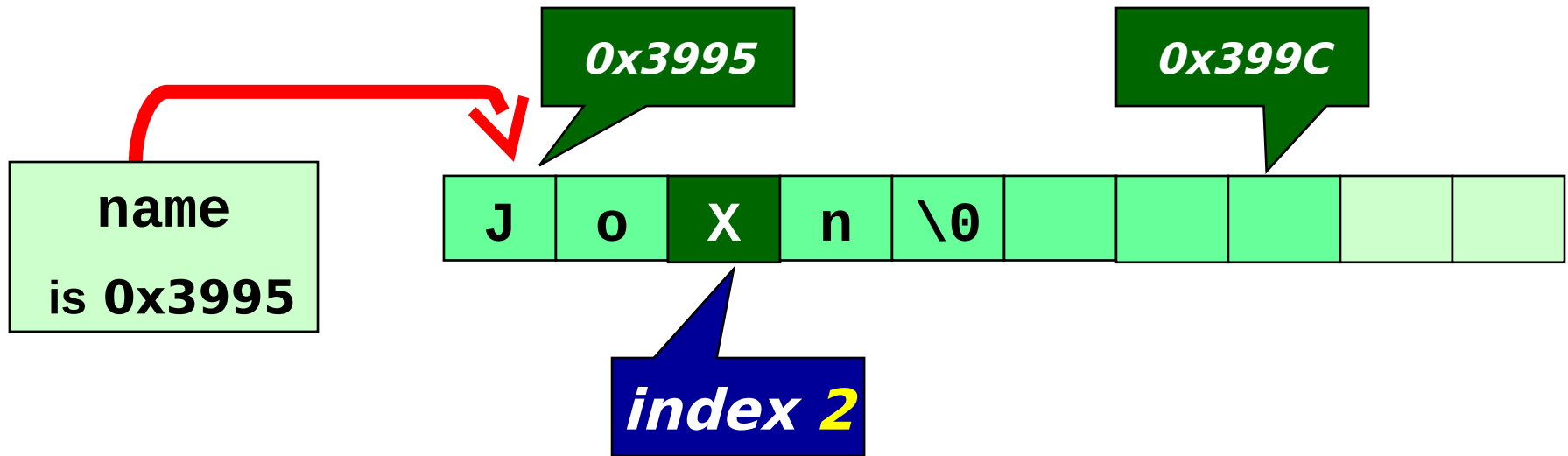
output: Char at index 2 is h.

A Char in a String (cont)



```
char name[8] = "John";  
name[2] = 'X';  
printf("Name: %s\n", name);
```

A Char in a String (cont)



```
char name[8] = "John";  
  
name[2] = 'X';  
printf("Name: %s\n", name);
```

output: Name: JoXn

String Operations

- **#include <string.h>**
- Operations:
 - Assignment: **strcpy()**
 - Concatenation: **strcat()**
 - Comparison: **strcmp()**
 - Length: **strlen()**

String Operation: Assignment

```
#include <stdio.h>
#include <string.h>

#define MAXLENGTH 100

int main()
{
    char string1[MAXLENGTH];
    char string2[MAXLENGTH];

    strcpy(string1, "Hello World!");
    strcpy(string2, string1);

    return 0;
}
```

```
string1: <garbage>
string2: <garbage>
```

String Operation: Assignment (cont)

```
#include <stdio.h>
#include <string.h>

#define MAXLENGTH 100

int main()
{
    char string1[MAXLENGTH];
    char string2[MAXLENGTH];

    strcpy(string1, "Hello World!");
    strcpy(string2, string1);

    return 0;
}
```

```
string1: "Hello World!"
string2: <garbage>
```

String Operation: Assignment (cont)

```
#include <stdio.h>
#include <string.h>

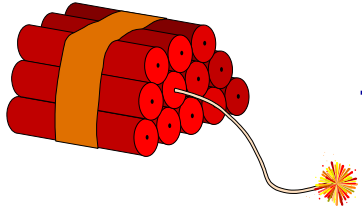
#define MAXLENGTH 100

int main()
{
    char string1[MAXLENGTH];
    char string2[MAXLENGTH];

    strcpy(string1, "Hello World!");
    strcpy(string2, string1);

    return 0;
}
```

```
string1: "Hello World!"
string2: "Hello World!"
```



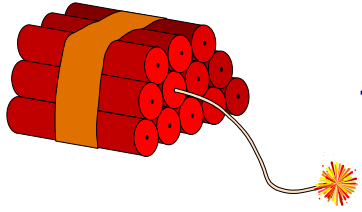
Common Mistake 1:

Incompatible types

Example:

```
char  name1[5] = "Ann";  
char  name2[5] = "Dave";  
  
name2 = name1;
```

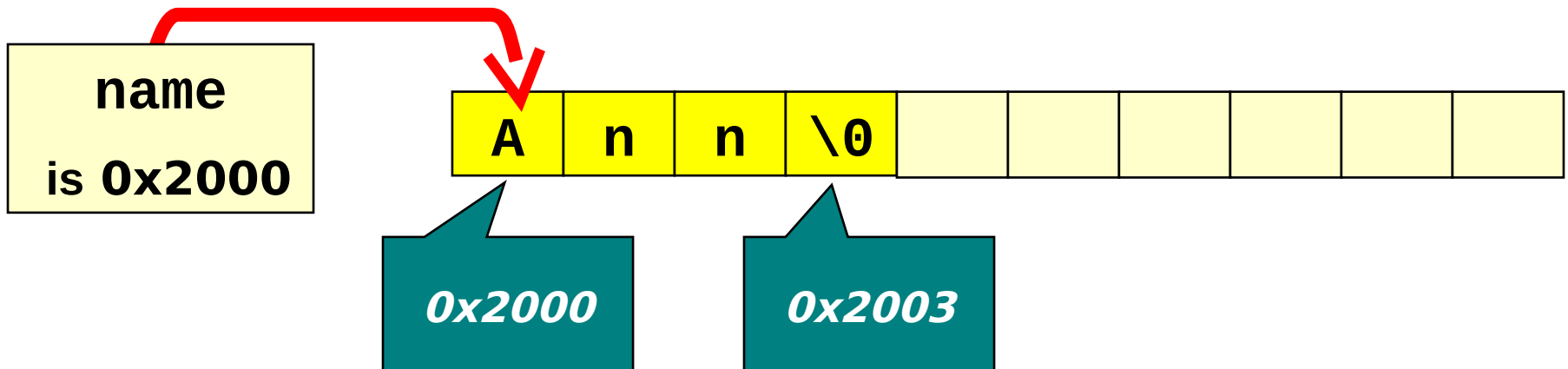
Error: "LValue required ..."

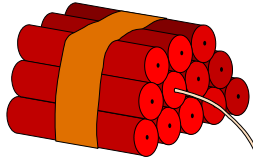


Common Mistake 2:

Not enough space

```
char  name[] = "Ann";  
strcpy(name, "David");
```

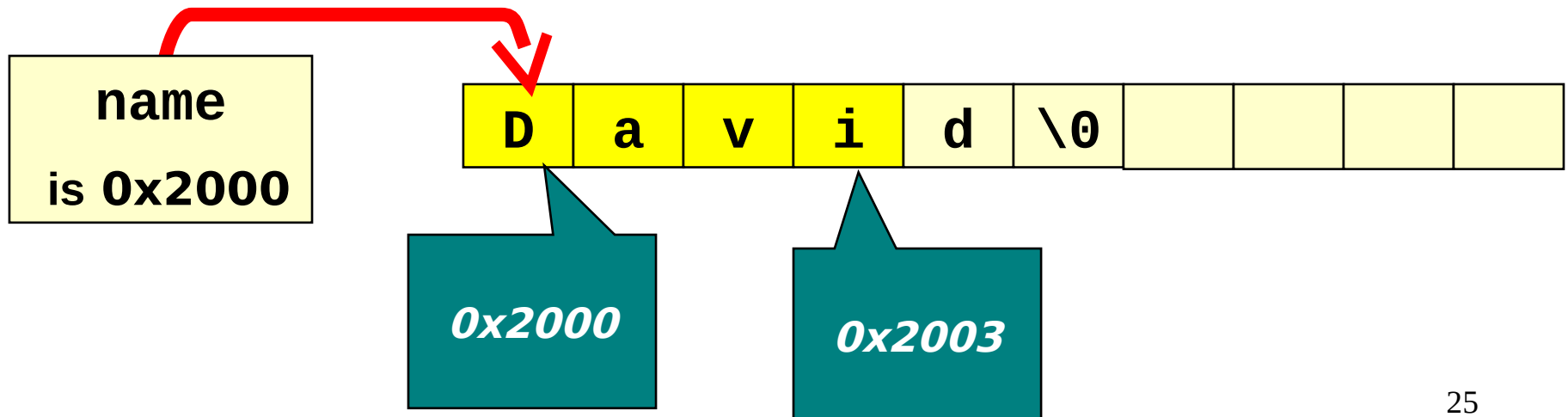




Common Mistake 2:

Not enough space

```
char name[] = "Ann";  
strcpy(name, "David");
```



String Operation: Concatenation

```
char string1[MAXLENGTH];  
char string2[MAXLENGTH];  
  
strcpy(string1, "Goodbye");  
strcpy(string2, ", Cruel ");  
  
strcat(string1, string2);  
strcat(string1, string2);  
strcat(string1, "World!");
```

```
string1: "Goodbye"  
string2: ", Cruel "
```

String Operation: Concatenation (cont)

```
char string1[MAXLENGTH];  
char string2[MAXLENGTH];  
  
strcpy(string1, "Goodbye");  
strcpy(string2, ", Cruel ");  
  
strcat(string1, string2);  
strcat(string1, string2);  
strcat(string1, "World!");
```

```
string1: "Goodbye, Cruel "  
string2: ", Cruel "
```

String Operation: Concatenation (cont)

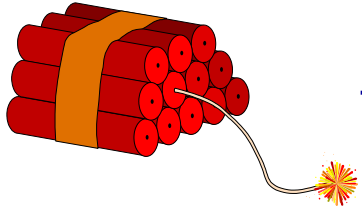
```
char string1[MAXLENGTH];  
char string2[MAXLENGTH];  
  
strcpy(string1, "Goodbye");  
strcpy(string2, ", Cruel ");  
  
strcat(string1, string2);  
strcat(string1, string2);  
strcat(string1, "World!");
```

```
string1: "Goodbye, Cruel , Cruel "  
string2: ", Cruel "
```

String Operation: Concatenation (cont)

```
char string1[MAXLENGTH];  
char string2[MAXLENGTH];  
  
strcpy(string1, "Goodbye");  
strcpy(string2, ", Cruel ");  
  
strcat(string1, string2);  
strcat(string1, string2);  
strcat(string1, "World!");
```

```
string1: "Goodbye, Cruel , Cruel World!"  
string2: ", Cruel "
```

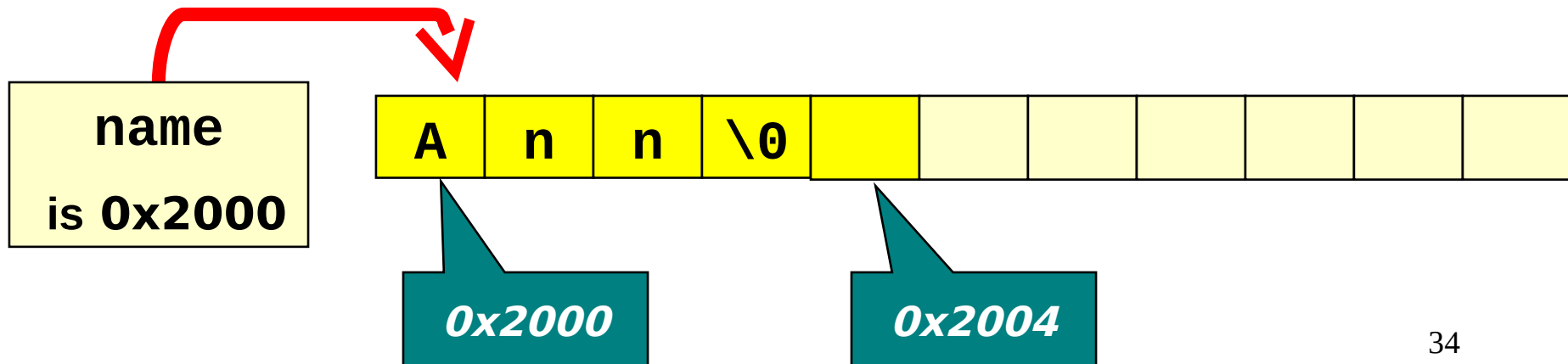


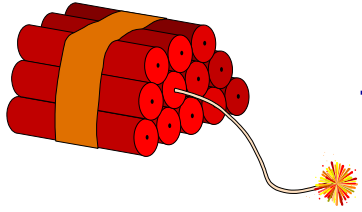
Common Mistake:

Not enough space

```
char name[5];
```

```
strcpy(name, "Ann");  
strcat(name, " Smith");
```

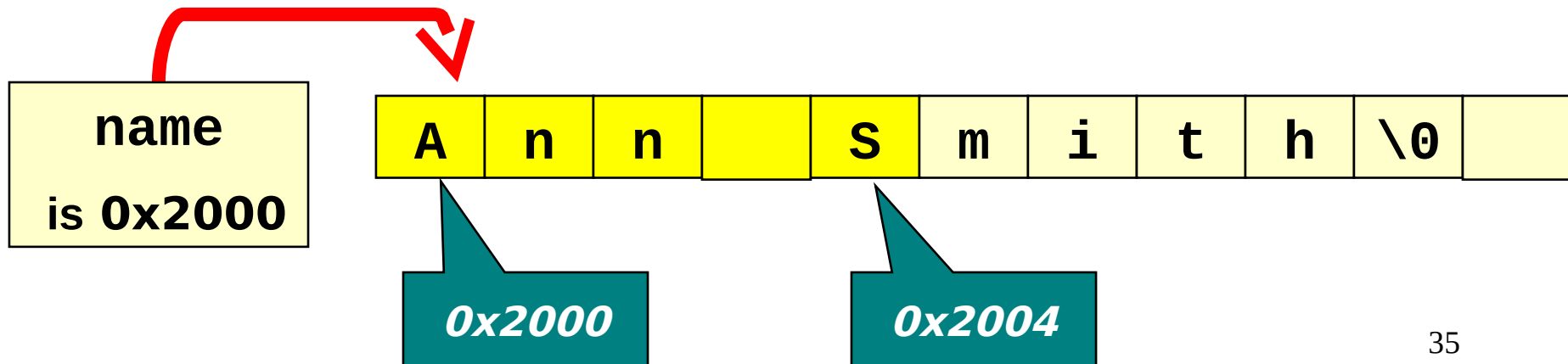




Common Mistake:

Not enough space

```
char  name[5];  
  
strcpy(name, "Ann");  
strcat(name, " Smith");
```



String Operation: Comparison

```
strcpy(string1, "Apple");  
strcpy(string2, "Wax");  
  
if (strcmp(string1, string2) < 0)  
{  
    printf("%s %s\n", string1, string2);  
}  
else  
{  
    printf("%s %s\n", string2, string1);  
}
```

Returns:

negative if $string1 < string2$

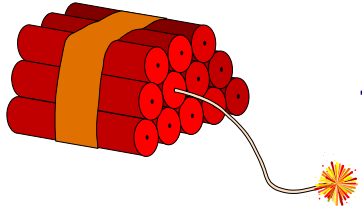
zero if $string1 == string2$

positive if $string1 > string2$

String Operation: Comparison (cont)

```
strcpy(string1, "Apple");  
strcpy(string2, "Wax");  
  
if (strcmp(string1, string2) < 0)  
{  
    printf("%s %s\n", string1, string2);  
}  
else  
{  
    printf("%s %s\n", string2, string1);  
}
```

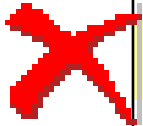
output: Apple Wax



Common Mistake:

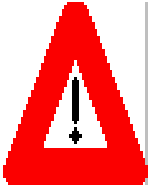
Wrong Comparison

```
strcpy(string1, "Apple");  
strcpy(string2, "Wax");
```



```
if (string1 < string2)  
{  
    printf("%s %s\n", string1, string2);  
}  
else  
{  
    printf("%s %s\n", string2, string1);  
}
```

Caution 1:



Not a Boolean

```
strcpy(string1, "Hi Mum");  
strcpy(string2, "Hi Mum");  
  
if ( strcmp(string1, string2) )  
{  
    printf("%s and %s are the same\n",  
           string1, string2);  
}
```

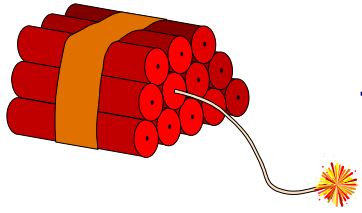
Returns zero if the strings are the same.

String Operation: Length

```
char string1[100];  
  
strcpy(string1, "Apple");  
  
printf("%d\n", strlen(string1));
```

output: 5

*Number of char-s
before the '\0'*



Common Mistake:

Not enough space

```
char name[5];
```

```
strcpy(name, "David");
```

*Don't forget
the '\0'*

name

is 0x3990



0x3990

0x3994

Character Strings as Parameters

- Strings as formal parameters are declared as **char*** or **char[]**

– *Examples:*

```
void Greet ( char* name )
```

```
void Greet ( char name[] )
```

- They point to the first element of the string (array of chars)
- Changes to the string inside the function affect the actual string

Example: hello3.c

```
#include <stdio.h>
#include <string.h>
#define NAMELEN 50

/* Print a simple greeting to
   the user */

void Greet ( char name[] )
{
    strcat(name, "! How are ya?");
}
```

```
int main()
{
    char user[NAMELEN];

    printf("Who are you? ");
    scanf("%s", user);
    Greet(user);
    printf("%s\n", user);

    return 0;
}
```



Example: hello3.c (cont)

```
#include <stdio.h>
#include <string.h>
#define NAMELEN 50

/* Print a simple greeting to
   the user */

void Greet ( char name[] )
{
    strcat(name, "! How are ya?");
}
```

```
int main()
{
    char user[NAMELEN];

    printf("Who are you? ");
    scanf("%s", user);
    Greet(user);
    printf("%s\n", user);

    return 0;
}
```



Example: hello3.c (cont)

```
#include <stdio.h>
#include <string.h>
#define NAMELEN 50

/* Print a simple greeting to
   the user */
```

```
void Greet ( char name[] )
{
    strcat(name, "! How are ya?");
}
```

```
int main()
{
    char user[NAMELEN];

    printf("Who are you? ");
    scanf("%s", user);
    Greet(user);
    printf("%s\n", user);

    return 0;
}
```

name

user



Jake! How are ya?\0

Example: hello3.c (cont)

```
#include <stdio.h>
#include <string.h>
#define NAMELEN 50

/* Print a simple greeting to
   the user */

void Greet ( char name[] )
{
    strcat(name, "! How are ya?");
}
```

```
int main()
{
    char user[NAMELEN];

    printf("Who are you? ");
    scanf("%s", user);
    Greet(user);
    printf("%s\n", user);

    return 0;
}
```

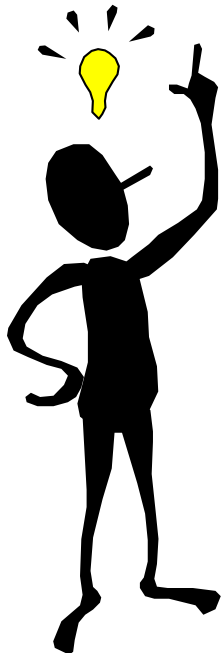
user



Jake! How are ya?\0

More of ***scanf*** demystified

*No ampersand
(&) in scanf
with strings!*



```
int main()
{
    char user[NAMELEN];

    printf("Who are you? ");
    scanf("%s", user);
    Greet(user);
    printf("%s\n", user);

    return 0;
}
```

Summary

- A string is a contiguous array of chars
- The string identifier is the address of the first char in the string
- Individual chars are accessed using the **str[index]** notation
- There are C library functions for copying, concatenating and comparing strings

Reading

- King
 - Chapter 13 (except Section 13.6)
- Deitel and Deitel
 - Chapter 8 (8.1 – 8.7)